

We use JSON to enable a better output:

```
db.employees.find().forEach(printjson);
```

...or use the `pretty()` function:

```
db.employees.find().pretty()
```

Fetching documents using a criteria

This is useful when there are specific documents you want to fetch.

Example: Equality criteria

To fetch the data of Ludacris from the `employees` collection, type:

```
db.employees.find({EmployeeName :
"Ludacris"}).pretty()
```

Example: Greater than criteria

The command syntax is:

```
db.collection_name.find({"field_
name":{"gt:criteria_value"}}.pretty()
```

To fetch details of employees aged over 25:

```
db.employees.find({"age":{"gt:25}}).
pretty()
```

Example: Less than criteria

The command syntax is as follows:

```
db.collection_name.find({"field_
name":{"lt:criteria_value"}}.pretty()
```

To fetch details of all employees who earn less than US\$ 4000:

```
db.employees.find({"Score":{"lt:4000}}).
pretty()
```

Example: Not equal to criteria

The command syntax is:

```
db.collection_name.find({"field_
name":{"ne:criteria_value"}}.pretty()
```

To get details of employees whose salary is not equal to US\$ 3500:

```
db.employees.find({"Salary":{"ne:3500}}).
pretty()
```

Update document

To use the `update()` method, the command syntax is:

```
db.collection_name.update(criteria,
update_data)
```

Example: Assuming I have a collection named `movies` in a database `bestmoviesdb`, the documents inside movies are:

```
> db.movies.find().pretty()
{
  "_id" : ObjectId("59bd2e9fce524
b733f14dd67"),
  "name" : "300",
  "genre" : thriller
}
{
  "_id" : ObjectId("59bd2ec5ce524
b733f14dd68"),
  "name" : "Avatar",
  "genre" : action
}
```

I now want to update the genre of *Avatar* to *Fantasy* from *Action*. The command syntax will be:

```
db.movies.update({"genre":"action"},{$s
et:{"genre":"fantasy"}})
```

Using the `save()` method to update, the command syntax is:

```
db.collection_name.save( {_
id:ObjectId(), new_document} )
```

To use the `save()` method, it is important to know the `unique _id` field of the document you want to update. If

one does not use the `_id` field here, the document is inserted as 'new' in the collection.

To get the `_id` of a document, run the following command:

```
db.get.find().pretty()
```

Once you have the `unique _id` of the field, run the following command to update it by using the `save()` method:

```
> db.movies.save({"_id" : ObjectId("59b
d2e73ce524b733f14dd65"), "name":
"Avatar", "genre":fantasy})
>
```

Community, support and the future of MongoDB

The MongoDB community is one of the most diverse, robust, active and vibrant there is. While the community was able to interact on other platforms such as Slack, Realm forums, etc, and it still can, as of mid-2020, MongoDB created a unified forum that incorporates all its users so they can share their experiences, opinions and interact with each other. This forum has been created keeping in mind that MongoDB users operate both offline and online, and so must have support for that.

To log in or sign into this forum, you can use your MongoDB cloud account or register for one if you do not have one; or log in with a Google account. Once you log in, you can then set up your profile and keep in touch with the topics important to you.

There is also a welcome bot called Leafy that can assist with navigating the registration steps. Make sure you connect with other MongoDB members and enthusiasts. **END** 🐧

By: Mir H.S. Quadri

The author is a scholar and researcher in the fields of artificial intelligence and machine learning. He has worked on multiple projects using a wide array of frameworks. He is also a FOSS enthusiast and actively contributes to several open source projects. You can find his work at codelatte.site.